UNIDAD 1 - SEMANA 3
RESUMEN DE CLASE

Nombre: Doménica Rojas
Ciclo: Quinto “A”
Asignatura: Sistemas Digitales

Tema: Temporizadores de hardware en el ESP32

1. Qué es un temporizador y por qué se utiliza

Un **temporizador (timer)** es un circuito del microcontrolador que cuenta pulsos de reloj de manera independiente de la CPU. La CPU lo configura y habilita; después, el periférico cuenta solo y puede avisar cuando llega a un valor programado.

Método	Cómo controla el tiempo	Limitación o ventaja
delay()	La CPU espera antes de continuar.	Bloquea el programa.
millis()	loop() consulta el tiempo transcurrido.	Puede reaccionar tarde si loop() está ocupado.
Timer de hardware	Un circuito cuenta y genera avisos.	No depende de que loop() pregunte.

2. Reloj, prescaler y resolución

En la explicación de clase, los temporizadores del ESP32 reciben un reloj **APB de 80 MHz** (80 millones de pulsos/s). Un **prescaler** divide esa frecuencia para obtener una cantidad más cómoda de *ticks* por segundo; cada tick incrementa el contador en 1.

- **Frecuencia de conteo:** f = 80 000 000 / N, donde N es el prescaler.
- **Resolución:** 1 / f segundos por tick; es la duración del incremento mínimo.
- **Valor de alarma:** alarma = tiempo deseado × f.
- **Comprobación:** tiempo = alarma × resolución.

Ejemplo de la clase: con N = 80, la frecuencia queda en **1 MHz**, por lo que cada tick dura **1 µs**. Para generar un aviso cada **0,5 s**, se configura una alarma de **500 000 ticks**.

Frecuencia	Alarma	Intervalo entre avisos
1 000 000 Hz	2 000	2 ms
1 000 000 Hz	500 000	0,5 s
10 000 Hz	2 500	0,25 s

3. Alarmas, autorrecarga y desbordamiento

La **alarma** es un valor elegido: cuando el contador lo alcanza, el timer genera un evento. Con **autorrecarga (auto-reload)**, el contador vuelve a 0 y repite el aviso periódicamente sin intervención del

programa. Esto es diferente del **desbordamiento**, que sucede al superar el máximo representable por sus bits.

El **ESP32 clásico** descrito en la presentación tiene **cuatro temporizadores de propósito general**, agrupados en TIMG0 y TIMG1, con contadores de **64 bits** y prescaler de 16 bits. El ATmega328P dispone de tres timers con contadores de 8 o 16 bits, por lo que sus límites son menores.

4. API de temporizadores e interrupciones

Una **API** agrupa funciones que permiten configurar el hardware sin escribir directamente en sus registros. Para trabajar con Arduino IDE se instala el **core "esp32 by Espressif Systems"**. Las diapositivas advierten que en la **versión 3.x** cambiaron las funciones y parámetros de temporizadores frente a la versión 2.x.

Cuando suena la alarma, el periférico puede producir una **interrupción**: la CPU pausa temporalmente lo que hacía, ejecuta una **ISR** (rutina de servicio de interrupción) y luego continúa. Una **callback** es la función que se registra para que el sistema la ejecute cuando ocurra el evento.

- **`IRAM_ATTR`:** solicita ubicar la rutina en la memoria de instrucciones rápida (IRAM).
- **`volatile`:** se aplica a las variables compartidas entre ISR y loop() para que se consulte su valor actualizado; por sí sola no garantiza atomicidad.
- **Patrón bandera:** la ISR solo marca que ocurrió algo; loop() revisa la bandera y realiza las operaciones lentas.